# Parte III – Database First

Gerando código e telas de CRUD a partir do banco do ReservaFlex com Prisma

| | |
|---|---|
| **Projeto** | ReservaFlex – sistema de reservas multi-segmento |
| **Equipe** | Luan, Érick, Vinicius e Gustavo |
| **Ferramenta** | Prisma 6.19.3 (Node.js + SQLite) |

Nessa parte a gente tinha que usar alguma ferramenta que gerasse as classes e o codigo a partir do banco de dados, e se desse ate o CRUD e as telas, tudo de forma automatica. Como o ReservaFlex ja tem o banco pronto, um SQLite com 8 tabelas que sao criadas pelo `schema.sql`, ele encaixou bem nesse tipo de abordagem.

## 1. O que é Database First

Database First é quando o desenvolvimento comeca pelo banco de dados. As tabelas, colunas, chaves e relacionamentos ja existem, e depois uma ferramenta le esse banco e gera o codigo da aplicação em cima dele, em vez da equipe escrever tudo na mão. O contrario disso é o Code First, onde a gente escreve as classes no codigo e a ferramenta é que cria o banco a partir delas.

Isso tem tudo a ver com reuso, porque o banco vira o artefato principal que é reaproveitado, todo o codigo de acesso aos dados nasce dele, e se alguma tabela mudar é so rodar a ferramenta de novo que o codigo é atualizado, sem precisar sair arrumando arquivo por arquivo.

## 2. Ferramenta escolhida: Prisma

A ferramenta que a gente escolheu foi o Prisma, que é um ORM pra Node.js. A escolha foi bem natural porque o ReservaFlex é feito em Node com SQLite, entao nao precisou mudar nada da tecnologia do projeto. O Prisma tem tres comandos que cobrem exatamente o que a atividade pede:

| Comando | O que a atividade pede | O que acontece |
|---|---|---|
| `npx prisma db pull` | Partir do banco de dados | Le o banco existente e escreve sozinho os models de cada tabela |
| `npx prisma generate` | Gerar classes e codigo | Transforma os models em codigo com os metodos de CRUD prontos |
| `npx prisma studio` | CRUD e telas automaticas | Abre no navegador telas pra listar, cadastrar, editar e excluir |

Usamos a versão 6.19.3, que é estavel e mais simples de configurar.

## 3. Passo a passo

### 3.1 Configuração e instalação

Antes de tudo foi criado o arquivo `prisma/schema.prisma` so com o endereço do banco, ou seja, nele ainda nao tinha NENHUMA tabela descrita:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = "file:../data/reservaflex.sqlite"
}
```

Depois instalamos o Prisma dentro do projeto:

```
npm.cmd install -D prisma@6.19.3
npm.cmd install @prisma/client@6.19.3
```

> No Windows o PowerShell bloqueou o comando `npm` por causa da politica de execução de scripts, por isso usamos o `npm.cmd` e o `npx.cmd`, que fazem a mesma coisa.

### 3.2 Lendo o banco (o Database First de verdade)

Esse é o passo principal. O comando `npx prisma db pull` abre o `reservaflex.sqlite`, le a estrutura das tabelas e escreve sozinho os models dentro do `schema.prisma`.

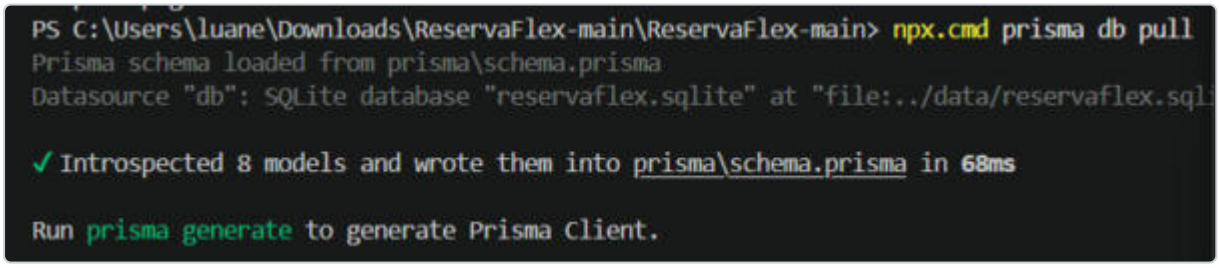



*Figura 1 – O Prisma leu o banco e gerou os 8 models ("Introspected 8 models")*

Ele encontrou as 8 tabelas do sistema (organizacao, usuario, recurso, reserva, regra_reserva, bloqueio_recurso, equipamento e configuracao_variabilidade) e montou cada uma com seus campos, tipos, valores padrão e chave primaria. Os relacionamentos tambem vieram direto das chaves estrangeiras, por exemplo no `bloqueio_recurso` ele ja ligou o `administrador_id` com o usuario e o `recurso_id` com o recurso:

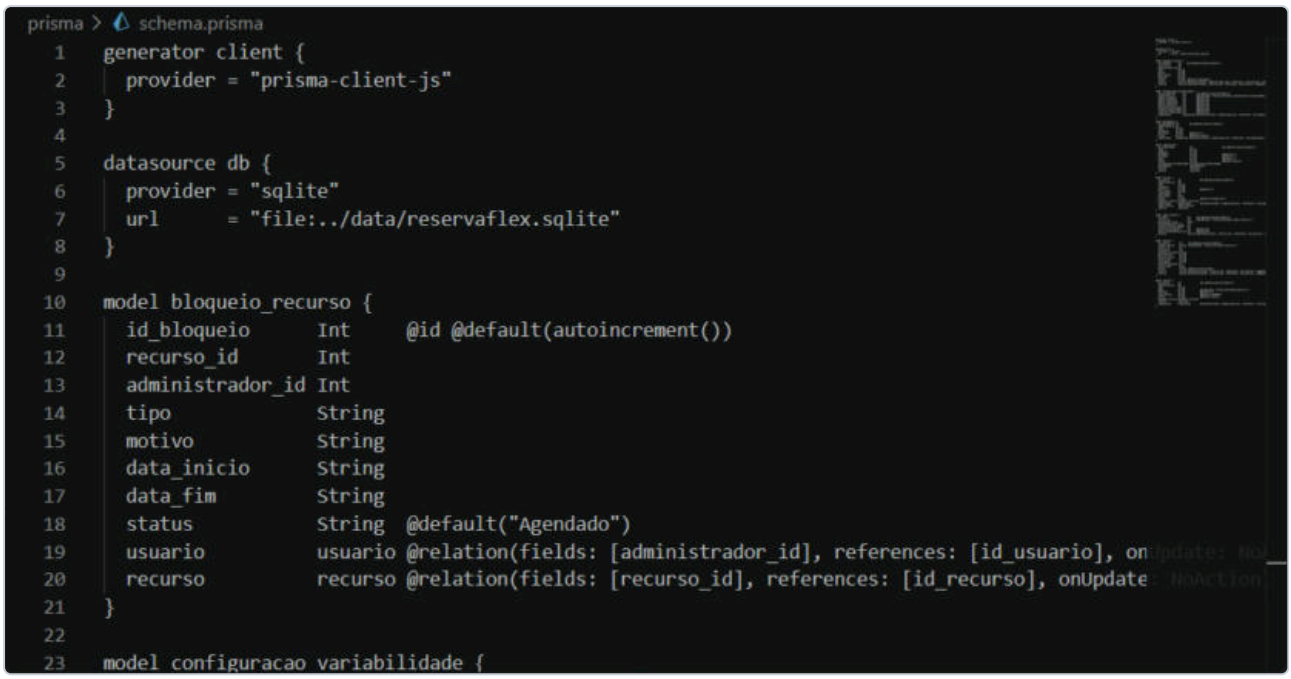



*Figura 2 – Trecho do schema.prisma depois do db pull, com os models escritos pela ferramenta*

Um detalhe que a gente percebeu é que as regras CHECK do SQL (tipo `capacidade > 0`) nao aparecem no schema do Prisma, porque ele nao representa esse tipo de restrição, mas elas continuam valendo dentro do banco.

### 3.3 Gerando o código

Com os models prontos, o comando `npx prisma generate` transforma eles em codigo JavaScript, o Prisma Client. Pra cada tabela ele gera os metodos de CRUD prontos, como `create`, `findMany`, `update` e `delete`, ja sabendo o tipo de cada campo.

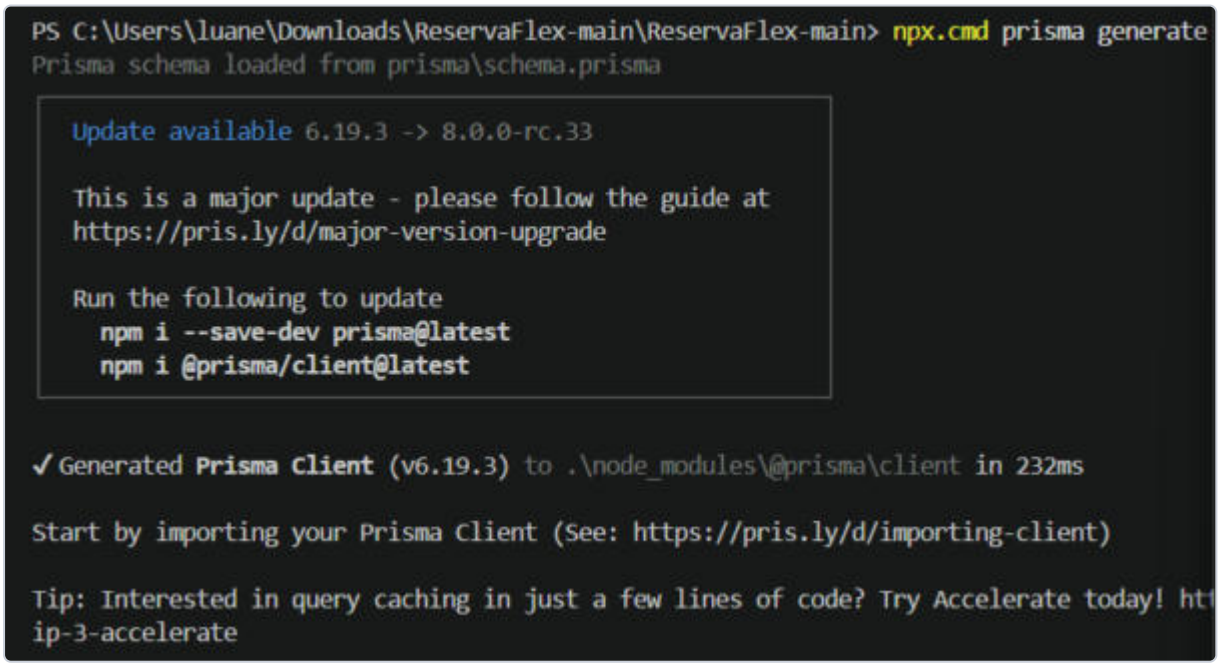



*Figura 3 – Prisma Client gerado a partir dos models*

O quadro de "Update available" que aparece no print é so um aviso de que existe uma versão mais nova do Prisma, nao é erro.

### 3.4 Usando o código gerado

Pra mostrar que o codigo gerado funciona de verdade, criamos o arquivo `demo-prisma.js`, que usa o Prisma Client sem nenhuma linha de SQL escrita na mão. Ele conta os registros de cada tabela, busca reservas ja trazendo junto o usuario e o recurso (o JOIN é o proprio Prisma que faz), e depois cria um equipamento de teste, atualiza o status dele e exclui, deixando o banco do jeito que estava. Um trecho do codigo:

```
const reservas = await prisma.reserva.findMany({
  take: 3,
  include: { usuario: true, recurso: true },
});

const novo = await prisma.equipamento.create({
  data: { organizacao_id: org.id_organizacao, nome: 'Projetor DEMO Prisma', categoria:
'Audiovisual' },
});
```

```
PS C:\Users\luane\Downloads\ReservaFlex-main\ReservaFlex-main> node demo-prisma.js

=== 1) READ: quantos registros tem em cada tabela ===
Organizacoes: 4
Usuarios:     6
Recursos:     8
Reservas:     2

=== 2) READ com relacionamento (JOIN feito pelo Prisma) ===
RF10483 | Luan (Integrante 1) reservou "Sala de Reunião Executive 04" em 2026-10-15
RF-1BB31BF7 | Responsável Arena Sports Club reservou "Quadra Society 01" em 2026-10-17

=== 3) CREATE / UPDATE / DELETE num equipamento de teste ===
Criado:      4 Projetor DEMO Prisma
Atualizado:  4 status = Inativo
Excluido:    4 (o banco volta como estava)
```

*Figura 4 – Execução do demo-prisma.js: leitura, JOIN e create/update/delete usando o código gerado*

### 3.5 Telas de CRUD automáticas

Por ultimo, o comando `npx prisma studio` abre no navegador (`localhost:5555`) uma interface com TODAS as tabelas do banco e telas de CRUD prontas. Da pra listar, cadastrar, editar e excluir registros sem ter programado nenhuma tela, e como ele usa o mesmo banco, o que é cadastrado ali aparece tambem no ReservaFlex.

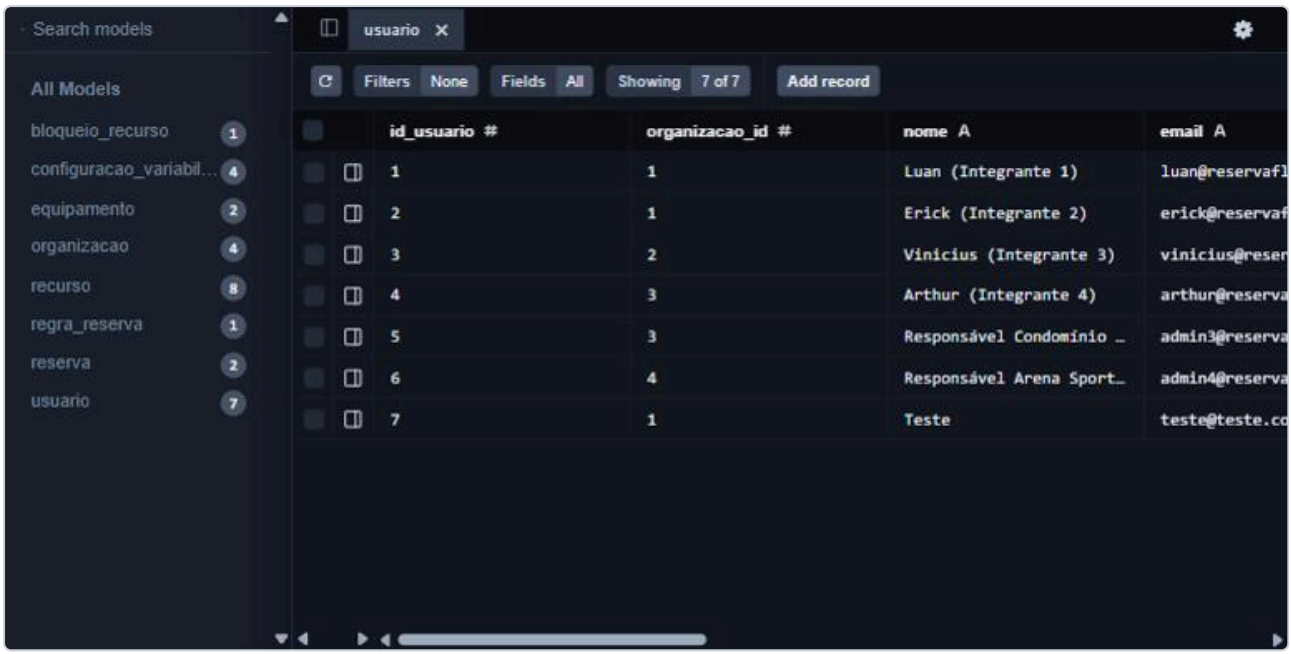



*Figura 5 – Prisma Studio com as 8 tabelas e a tabela usuario aberta*

## 4. Como vai ser a demonstração

Na apresentação um membro da equipe vai mostrar o Database First funcionando nessa ordem:

1. Mostrar o `schema.sql` e o banco, deixando claro que o banco ja existia antes de tudo.
2. Mostrar o `schema.prisma` so com o endereço do banco e rodar o `npx prisma db pull`, ai os models aparecem sozinhos.
3. Rodar o `npx prisma generate` pra gerar o codigo com o CRUD.
4. Rodar o `node demo-prisma.js` mostrando o codigo gerado funcionando sem SQL na mão.
5. Abrir o `npx prisma studio` e cadastrar um registro, depois mostrar que ele aparece no ReservaFlex.

## 5. Conclusão

Com o Database First a gente conseguiu gerar o modelo, o codigo de acesso e as telas de CRUD do ReservaFlex so a partir do banco que ja existia, sem escrever SQL nem tela na mão. Pra uma linha de produto isso ajuda MUITO, porque se aparecer um segmento novo (tipo uma academia ou um hotel) e o banco ganhar tabelas novas, é so rodar o db pull e o generate de novo que o codigo acompanha.

A limitação é que as telas do Prisma Studio sao mais de administração, entao nao substituem as telas do sistema pro usuario final, e algumas regras do banco como o CHECK ficam so no SQL e nao aparecem no modelo gerado.